

DSS Radar Program Customization Manual

By Petri Miettinen

Requirements:

- Setup DSS Radar Program According to setup Manual
- AWR2944EVM Board
- 3.5mm power plug with 12V and less than 2.5A output
- USB-A to Micro-USB Cable

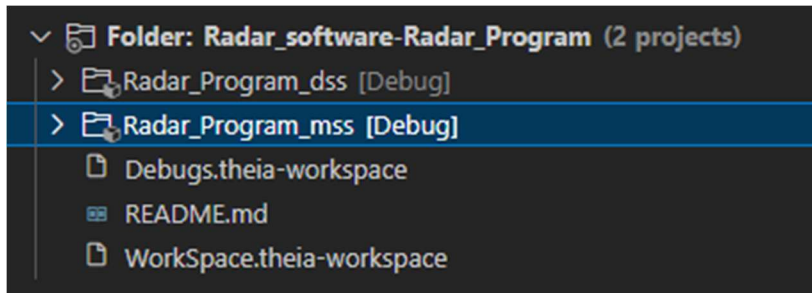
Introduction

This manual aims to run through the basic steps of setting up a LED blinker demo functionality as part of the existing DSS Radar Program. These steps aims to provide the outline to follow for creating custom functions that are integrated as part of the Task Scheduler of the DSS Radar Program, as well as expanding on parts of other options within Code Composer Studio for customizations.

Led Demo

Starting with the LED demo function, the initial steps will simply run through the steps to make the demo work, with further explanation for the steps taken following after the implementation.

Step 1. Initial CCS Setup + View



If all steps during the original manual were executed correctly, the above picture should represent the view in the CCS Explorer.

Follow this process:

1. Navigate to Radar_program_mss > mss > mss_main.c

This is the main folder of the program and where the LED Demo will be implemented.

2. Add the following:
 - a. Around line 78: `StackType_t gLedTskStack[1024U] __attribute__((aligned(32), section(".bss.dll.l3")));`

This creates a memory array used for the task that will be created later.

3. Next navigate to mmw_mss.h
4. Near line 300 find section "typedef struct mmwDemo_TaskHandles_t"
5. Under this add the following:
 - a. `TaskHandle_t ledTask`
 - b. `StaticTask_t ledTaskObj`

These variables will be used for pointers later in the process.

This concludes the steps for setting up the function.

Step 2. The Actual Function

For the demo purposes this LED function is a very simple one and can be written down somewhere within `mss_main.c` or in its own header:

```
void led_task(void *args)
{
    while (1)
    {
        GPIO_pinWriteHigh(GPIO_LED_BASE_ADDR, GPIO_LED_PIN);
        vTaskDelay(pdMS_TO_TICKS(500));
        GPIO_pinWriteLow(GPIO_LED_BASE_ADDR, GPIO_LED_PIN);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

The radar program file already has the base addresses and pins assigned, so there should be no further setup needed.

Step 3. Setting up the Function Task

The method the Radar Program works is using FreeRTOS task scheduling system. What this means, is the function that is written can be integrated into a task schedule that will manage the timing, space management and other tasks.

The breakdown of these task functions is provided after this tutorial, in the meantime follow these steps:

1. Navigate back to mss_main.c
2. Around line 142 find MmwDemo_CLInit(MMWDEMO_CLI_TASK_PRIORITY);
 - a. The task function needs to be implemented after this line and before SemaphoreP_pend;
3. Implement the following functions:

```
gMmwMssMCB.taskHandles.ledTask =  
xTaskCreateStatic(  
    ledTask,  
    "Led_task",  
    512,  
    NULL,  
    1,  
    gLedTskStack,  
    &gMmwMssMCB.taskHandles.ledTaskObj);  
  
configASSERT(gMmwMssMCB.taskHandles.ledTask != NULL);
```

Step 4. Build and Test

Following this, rebuild the code and following the steps for uploading the code detailed in the DSS Radar Program Manual, upload the code to the AWR2944EVM. Immediately the LED on the board should begin blinking.

Explanation Breakdown

The tutorial provided the rough steps to implement a very basic function as part of the existing code. This section seeks to explain some of the steps taken for the implementation.

The FreeRTOS task header provides the following breakdown for the task creation.

```
TaskHandle_t xTaskCreateStatic( TaskFunction_t  
pvTaskCode,  
                                const char * const pcName,  
                                uint32_t ulStackDepth,  
                                void *pvParameters,  
                                UBaseType_t uxPriority,  
                                StackType_t *pxStackBuffer,  
                                StaticTask_t *pxTaskBuffer );
```

Create a new task and add it to the list of tasks that are ready to run.

Internally, within the FreeRTOS implementation, tasks use two blocks of memory. The first block is used to hold the task's data structures. The second block is used by the task as its stack. If a task is created using `xTaskCreate()` then both blocks of memory are automatically dynamically allocated inside the `xTaskCreate()` function. (see <https://www.FreeRTOS.org/a001111.html>). If a task is created using `xTaskCreateStatic()` then the application writer must provide the required memory. `xTaskCreateStatic()` therefore allows a task to be created without using any dynamic memory allocation.

@param pvTaskCode Pointer to the task entry function. Tasks must be implemented to never return (i.e. continuous loop).

@param pcName A descriptive name for the task. This is mainly used to facilitate debugging. The maximum length of the string is defined by `configMAX_TASK_NAME_LEN` in `FreeRTOSConfig.h`.

@param ulStackDepth The size of the task stack specified as the number of variables the stack can hold - not the number of bytes. For example, if the stack is 32-bits wide and `ulStackDepth` is defined as 100 then 400 bytes will be allocated for stack storage.

@param pvParameters Pointer that will be used as the parameter for the task being created.

@param uxPriority The priority at which the task will run.

@param pxStackBuffer Must point to a `StackType_t` array that has at least `ulStackDepth` indexes - the array will then be used as the task's stack, removing the need for the stack to be allocated dynamically.

@param pxTaskBuffer Must point to a variable of type `StaticTask_t`, which will then be used to hold the task's data structures, removing the need for the memory to be allocated dynamically.

@return If neither `pxStackBuffer` or `pxTaskBuffer` are `NULL`, then the task will be created and a handle to the created task is returned. If either `pxStackBuffer` or `pxTaskBuffer` are `NULL` then the task will not be created and `NULL` is returned.

Trying to summarize as follows:

PvTaskFunction : The name of the function for the task. Point it at whatever function contains your task's logic. That function must run forever (usually a while(1) loop) and never return.

PcName : Just a label/nickname for the task, purely so it's readable in a debugger or logs. Doesn't affect behavior. Keep it under whatever configMAX_TASK_NAME_LEN is set to.

ulStackDepth : How much stack memory to give the task, but expressed in words, not bytes. On a 32-bit MCU (like your AWR2944's Cortex-R5F), each "word" is 4 bytes. So ulStackDepth = 100 actually reserves 400 bytes. Easy to trip over if you're thinking in bytes.

pvParameters : A generic pointer you can use to pass data into the task when it starts (e.g. a config struct, an ID, whatever). Shows up as the argument inside your task function. Pass NULL if you don't need to pass anything.

uxPriority : Sets how "important" the task is to the scheduler — higher number = higher priority = more likely to preempt lower-priority tasks. Range is bounded by configMAX_PRIORITIES in your FreeRTOSConfig.h (the Led demo priority is 1, therefore it is the lowest priority to anything else in the code)

pxStackBuffer: Since this is the static version of task creation (no heap allocation), you have to supply the actual memory for the stack yourself: an array of StackType_t with at least ulStackDepth elements. This replaces what would normally be malloc'd behind the scenes.

pxTaskBuffer : Same idea, but for the task's internal control structure (the TCB — Task Control Block) instead of its stack. You provide a StaticTask_t variable, and FreeRTOS fills it in rather than allocating it dynamically.

Return value — If you gave valid (non-NULL) buffers for both stack and TCB, you get back a task handle you can use later (e.g. to suspend/delete/notify the task). If either buffer was NULL, task creation silently fails and you get NULL back — worth checking, since a NULL handle used later will cause a hard fault.

Summary

In simplest terms for custom functions, you implement a looping function with a while(1) loop, create pointers that the taskCreate refers to for allocating static memory for the task as well as its priority for how it runs with the rest of the overall program.

This explanation is very surface level, but hopes to provide a decent starting point.

Other useful information

The ifdef definitions

Observing the code you might notice spots like “`#ifdef ENET-STREAMING`” bracketed then with an `#endif`

These are familiar as definitions normal C code also provides, but the notable difference is that the main code itself does not have anything like “`#define ENET-STREAMING 1`” which might decide the ifdef functionality.

Instead, the CCStudio handles these internally through a “predefined symbols” category which can be accessed as follows:

For the **Radar_Program_dss**:

1. Right click the project folder.
2. Click **Properties**
3. Expand **C6000 Compiler**
4. Choose **Predefined Symbols**

The window has section for Pre-define and un-define.

Pressing the + symbol on the Pre-Define section and adding a text value eg. “EXAMPLE” is essentially the same as writing “`#Define EXAMPLE 1`”, but functionally across all the libraries and source files included in the project file.

For Radar_Program_mss the approach is essentially the same but instead of C6000 Compiler the option under the properties is called **Arm Compiler**

